Data Structure Lab7 : Stack 2022-2023

Topics

1. Create Stack Interface
2. Create Stack Using Array
3. Create Stack Using Linked Lists
4. Implement Basic Methods of Stack
    - isEmpty()
    - size()
    - top()
    - push(E e)
    - pop()

Homework

1. Implement a method with signature transfer(S, T) that transfers all elements from stack S onto stack T, so that the element that starts at the top of S is the first to be inserted onto T, and the element at the bottom of S ends up at the top of T.

Data Structure Lab7 : Stack 2022-2023

```
} <public class Lab7<T
;private Node<T> top

} <private static class Node<T
;T value
;Node<T> next

} public Node(T value)
;this.value = value
;this.next = null
{
{

} ()public Lab7
;top = null
{

// السؤال \
} public void transfer(Lab7<T> S, Lab7<T> T)
} while (S.top != null)
;T.push(S.pop())
{
{

} public void push(T value)
;Node<T> newNode = new Node<>(value)
;newNode.next = top
;top = newNode
{

} ()public T pop
;if (top == null) return null
;T value = top.value
;top = top.next
;return value
{

} ()public boolean isEmpty
;return top == null
```

Data Structure Lab7 : Stack 2022-2023

2. Give a recursive method for removing all the elements from a stack.

   public void removeAll() { if (top != null) { T temp = pop(); removeAll(); } }

3. Postfix notation is an unambiguous way of writing an arithmetic expression without parentheses. It is defined so that if “(exp1)op(exp2)” is a normal fully parenthesized expression whose operation is op, the postfix version of this is “pexp1 pexp2 op”, where pexp1 is the postfix version of exp1 and pexp2 is the postfix version of exp2. The postfix version of a single number or variable is just that number or variable. So, for example, the postfix version of “((5 + 2) ∗ (8 − 3))/4” is “5 2 + 8 3 − ∗ 4 /”. Describe a nonrecursive way of evaluating an expression in postfix notation.

Data Structure Lab7 : Stack 2022-2023

```
} public int evaluatePostfix(String expression)
            ;()<>Lab7<Integer> stack = new Lab7
        ;(" ")String[] tokens = expression.split

                } for (String token : tokens)
                   } if (isOperator(token))
                      ;()int b = stack.pop
                      ;()int a = stack.pop
;stack.push(performOperation(a, b, token))
                                          } else{
      ;stack.push(Integer.parseInt(token))
                                                 {
                                                   {

                                        ;()return stack.pop
                                                       {

                                  } private boolean isOperator(String token)
;("/")return token.equals("+") || token.equals("-") || token.equals("*") || token.equals
                                                       {

                        } private int performOperation(int a, int b, String operator)
                                                  } switch (operator)
                                                         :"+" case
                                                  ;return a + b
                                                         :"-" case
                                                  ;return a - b
                                                         :"*" case
                                                  ;return a * b
                                                         :"/" case
                                                  ;return a / b
                                                        :default
                        ;throw new IllegalArgumentException("Invalid operator")
                                                               {
                                                                 {
```

4. Implement the clone( ) method for the ArrayStack class.

Data Structure Lab7 : Stack 2022-2023

```
} ()public Lab7<T> clone
;()<>Lab7<T> clonedStack = new Lab7
 ;()<>Lab7<T> tempStack = new Lab7

      } while (!this.isEmpty())
  ;tempStack.push(this.pop())
                              {

  } while (!tempStack.isEmpty())
    ;()T value = tempStack.pop
            ;this.push(value)
     ;clonedStack.push(value)
                              {

            ;return clonedStack
                                }
```

5. Implement a program that can input an expression in postfix notation (see Exercise C-6.19) and output its value

```
} public int evaluatePostfixFromInput(String expression)
            ;return evaluatePostfix(expression)
                                                {
                                                  {
```